

STAR Robot Control System

Comprehensive Technical Documentation

Complete Reference for Software Engineers

Version 1.0
December 2025

STAR Development Team
Locked Inc.

Contents

1	Nanopb (Protobuf) Protocol with ARQ and CRC for SPI Communication	3
1.1	Executive Summary	3
1.1.1	Control Loop Frequency Rationale	3
1.2	Key Design Decisions	4
1.2.1	Protocol Stack	4
1.2.2	Byte Order and Standards Compliance	4
1.2.3	Memory Budget	5
1.3	Implementation Requirements	6
1.3.1	Protocol Stack Architecture	6
1.3.2	RX72N (SPI Peripheral) Implementation	6
1.3.3	RPi5 (SPI Controller) Implementation	7
1.3.4	ARQ Symmetry and State Management	8
1.3.5	Example Transaction: RPi5 Sends Velocity Command	9
1.4	ARQ Communication Flow	9
1.4.1	Normal Operation (No Retransmission)	10
1.4.2	Error Recovery Scenarios	10
1.5	Error Handling Strategies	12
2	Protocol Buffer Schema Architecture	13
2.1	Architecture Overview	13
2.2	Service Specifications	13
2.2.1	MotorControlService	13
2.2.2	TelemetryService	14
2.2.3	BatteryManagementService	15
2.2.4	ConfigurationService	15
2.2.5	FirmwareUpdateService	16
2.3	common.proto – Shared Types	16
2.4	Code Generation and Integration	17
3	Hardware Pin Connections	18
3.1	RX72N Microcontroller Pinout (R5F572NNHGFP#30)	18
3.1.1	Package Information	18
3.1.2	Motor Control Pin Assignments	18
3.1.3	Communication Interfaces	19
3.1.4	Status and Safety	20
3.1.5	System Clocks	20
3.2	Raspberry Pi 5 Pinout	20
3.2.1	RPi5 Pin Function Categories	21
3.3	Additional Peripheral Connections	22
3.3.1	IMU (Inertial Measurement Unit)	22
3.3.2	Stereo Cameras	22
3.3.3	Temperature Sensor	23
4	Protocol Buffer Style Guide	24
4.1	Terminology Standard	24
4.2	Protocol Buffer Naming Conventions (Boston Dynamics Style)	24

4.3	Unit Suffixes (MKS System)	24
4.4	General Documentation Standards	25
4.5	NASA Power of 10: Rules for Safety-Critical Code	25
4.5.1	Compliance Status Legend	25
4.5.2	Rule 1: Simplify Control Flow	25
4.5.3	Rule 2: Fixed Loop Upper-Bounds	26
4.5.4	Rule 3: No Dynamic Memory After Initialization	26
4.5.5	Rule 4: Keep Functions Short	27
4.5.6	Rule 5: Use Assertions Generously	27
4.5.7	Rule 6: Declare Data at Smallest Scope	28
4.5.8	Rule 7: Check All Return Values and Parameters	28
4.5.9	Rule 8: Limit Preprocessor Use	29
4.5.10	Rule 9: Restrict Pointer Use	30
4.5.11	Rule 10: Compile with Maximum Warnings	30
4.5.12	Summary: STAR Compliance Matrix	31
4.5.13	References	31
5	Gateway Architecture	32
5.1	Overview	32
5.2	Protocol Stack Implementation	32
5.3	Directory Structure	32
5.4	Frame Package	33
5.4.1	Frame Constants	33
5.4.2	Frame Types	33
5.5	Transport Package	33
5.5.1	SPI Configuration	34
5.5.2	Transport Interface	34
5.6	ARQ Package	34
5.6.1	ARQ Parameters	34
5.6.2	ARQ State Machine	34
5.7	gRPC Services	34
5.8	Build and Test	35
5.9	Dependencies	35

1 Nanopb (Protobuf) Protocol with ARQ and CRC for SPI Communication

1.1 Executive Summary

Overview: Protocol Buffers (nanopb) encapsulated in frames with CRC-32 and Automatic Repeat Request (ARQ) for reliable SPI communication between RPi5 and RX72N.



System Constraints:

- Renesas RX72N: 4MB flash, 1MB SRAM
- 100Hz SPI communication (10ms period)
- 250Hz motor control loop (4ms period)
- Static allocation only (no malloc)

1.1.1 Control Loop Frequency Rationale

Why different loop rates?

The nested loop architecture follows classic control systems design. The outer loop (SPI at 100Hz) handles command distribution and telemetry collection, while the inner loop (PID at 250Hz) responds quickly to motor dynamics. Running PID at 2.5 times the communication rate ensures smooth control between commands without wasting CPU on excessive computation.

Why 250Hz for motor control?

This is a Nyquist sampling consideration. With DC gearmotors at 210 RPM (3.5 Hz mechanical speed) and a first-order time constant $\tau = 75$ ms (cutoff frequency $f_c = 1/(2\pi\tau) \approx 2.1$ Hz), the 250Hz control rate provides approximately 60 times oversampling of the mechanical dynamics - well above the 2 times Nyquist minimum.

Running at 250Hz balances:

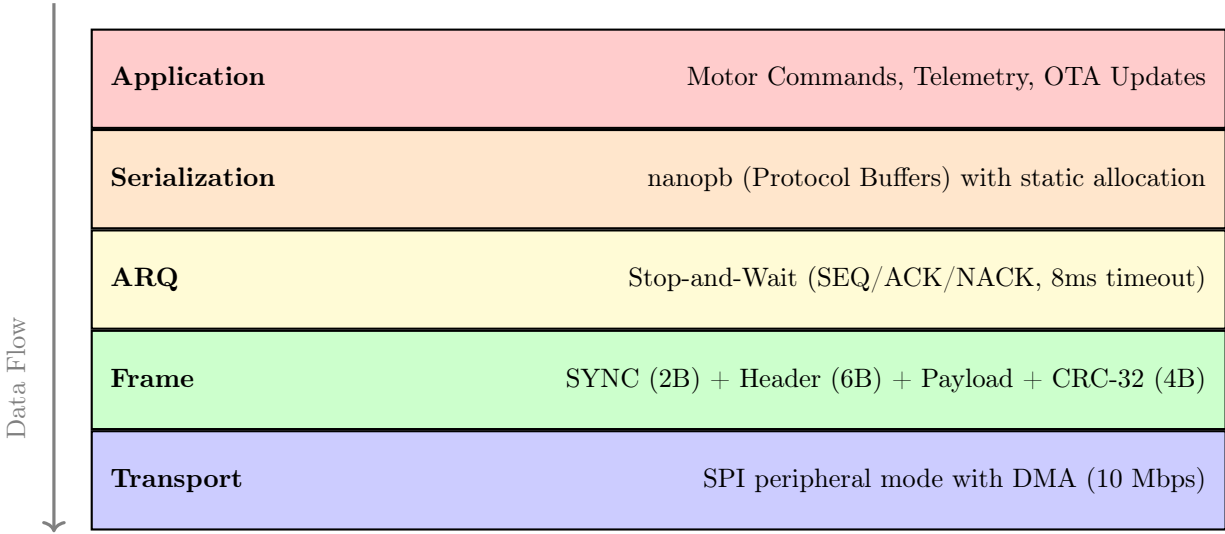
- Adequate sampling of motor dynamics (60 times oversampling)
- Efficient CPU utilization (4 motors $\times$ PID + encoder reads)
- Low latency between velocity commands and motor response
- Power efficiency (fewer ADC conversions, GPIO reads, and computations)

For context, control loop frequencies should match system dynamics:

- DC gearmotors: 50-500 Hz typical
- High-speed servos: 1-10 kHz
- Motor current control: 10-50 kHz
- Temperature control: 0.1-10 Hz

1.2 Key Design Decisions

1.2.1 Protocol Stack



1.2.2 Byte Order and Standards Compliance

The protocol follows established standards for byte ordering to ensure cross-platform compatibility:

Table 1: Byte Order Standards by Protocol Field

Field	Byte Order	Standard	Rationale
SYNC	Big-endian	RFC 1700	Network byte order for headers
SEQ	Big-endian	RFC 1700	Network byte order for headers
LEN	Big-endian	RFC 1700	Network byte order for headers
TYPE	N/A (1 byte)	–	Single byte, no endianness
FLAGS	N/A (1 byte)	–	Single byte, no endianness
CRC-32	Little-endian	IEEE 802.3	Standard CRC-32 polynomial

SYNC Magic Word (0x55AA): The frame synchronization marker uses the distinctive bit pattern 0x55AA (binary: 0101010101010101 in big-endian wire format). This alternating pattern is easily recognizable and unlikely to appear in random data, making it ideal for frame alignment. The value 0x55AA is transmitted as bytes [0x55, 0xAA] on the wire.

RFC 1700 (Network Byte Order): All multi-byte header fields (SYNC, SEQ, LEN) use big-endian format following RFC 1700 “Assigned Numbers” standard. This ensures compatibility when communicating between different architectures (ARM RPi5 and Renesas RX72N, both little-endian internally).

IEEE 802.3 (CRC-32): The frame checksum uses IEEE 802.3 polynomial (0x04C11DB7) in little-endian format, compatible with standard Ethernet CRC libraries. This allows zero-copy CRC validation using hardware accelerators where available.

Key Insight: Mixed byte orders are intentional — headers follow network standards for cross-platform compatibility, while CRC follows IEEE 802.3 for hardware acceleration and library compatibility.

1.2.3 Memory Budget

Table 2: SRAM Usage (512 KB total)

Component	Size (KB)	Usage (%)
DMA double-buffer	8.0	1.56
RX/TX buffers	2.0	0.39
Protobuf structs	1.5	0.29
ARQ state	4.0	0.78
Total	15.5	3.03

Table 3: Flash Usage (16 MB total)

Component	Size (KB)	Usage (%)
Generated protobuf	80	0.49
Nanopb runtime	12	0.07
Frame + ARQ layers	8	0.05
Total	100	0.61

Memory Budget Calculations:

SRAM (512 KB total):

- **DMA double-buffer:** 2×4092 bytes = 8184 bytes = 8.0 KB
Calculation: Two ping-pong DMA buffers at `STAR_SPI_DMA_MAX_SIZE` (4092 bytes each)
Usage: $\frac{8.0}{512} \times 100 = 1.56\%$
- **RX/TX buffers:** $1024 + 1024 = 2048$ bytes = 2.0 KB
Calculation: 1 KB RX buffer + 1 KB TX buffer for staging protobuf messages
Usage: $\frac{2.0}{512} \times 100 = 0.39\%$
- **Protobuf structs:** 1536 bytes = 1.5 KB
Calculation: Largest message structures (SetVelocityResponse $\approx$ 600 bytes, TelemetryData $\approx$ 400 bytes)
Usage: $\frac{1.5}{512} \times 100 = 0.29\%$
- **ARQ state:** $4 + 4 + 8 + 4 + 4092 = 4112$ bytes $\approx$ 4.0 KB
Calculation: Sequence number (4B) + retry counter (4B) + timestamp (8B) + state (4B) + retry buffer (4092B)
Usage: $\frac{4.0}{512} \times 100 = 0.78\%$
- **Total SRAM:** $8.0 + 2.0 + 1.5 + 4.0 = 15.5$ KB
Total usage: $\frac{15.5}{512} \times 100 = 3.03\%$

Flash (16 MB = 16384 KB total):

- **Generated protobuf:** 80 KB
Calculation: 6 proto files (common, motor_control, telemetry, battery_management, configuration, firmware_update) $\times$ 13 KB avg compiled size
Usage: $\frac{80}{16384} \times 100 = 0.49\%$
- **Nanopb runtime:** 12 KB
Calculation: Compiled size of pb_encode.c, pb_decode.c, pb_common.c
Usage: $\frac{12}{16384} \times 100 = 0.07\%$
- **Frame + ARQ layers:** 8 KB
Calculation: Framing logic + CRC-32 + ARQ state machine + retransmission logic
Usage: $\frac{8}{16384} \times 100 = 0.05\%$
- **Total Flash:** $80 + 12 + 8 = 100$ KB
Total usage: $\frac{100}{16384} \times 100 = 0.61\%$

1.3 Implementation Requirements

1.3.1 Protocol Stack Architecture

The protocol is implemented as a layered stack on both the RPi5 (SPI controller) and RX72N (SPI peripheral). Each layer has specific responsibilities:

Layer	Responsibility	Implementation
5. Application	Motor commands, telemetry, control logic	Different per platform
4. Protobuf	Message encode/decode (nanopb)	Both sides
3. ARQ	Sequence numbering, ACK/NACK, retry logic	Both sides (symmetric)
2. Framing	Header/footer, CRC-32 verification	Both sides (identical)
1. SPI+DMA	Physical transport with double-buffering	Both sides (different roles)

Table 4: Protocol stack layers and implementation requirements

Key Insight: Layers 2-4 are nearly identical on both platforms. Only the application layer (Layer 5) and SPI role (Layer 1: controller vs peripheral) differ.

1.3.2 RX72N (SPI Peripheral) Implementation

Layer 1: SPI Peripheral with DMA

- Configure RSPI in peripheral mode
- Set up DMA with double-buffering: 2×4092 bytes
- Register DMA interrupt callbacks for buffer swapping
- Wait for chip select from RPi5, respond to clock

Layer 2: Frame Layer (Custom Code)

RX Path:

- Parse frame header (magic bytes, length, sequence number)
- Verify CRC-32 over header + payload
- If CRC fails $\rightarrow$ discard packet, send NACK
- If CRC valid $\rightarrow$ extract payload, pass to ARQ layer

TX Path:

- Build frame: [Header | Payload | CRC32 | Footer]
- Calculate CRC-32 over header + payload
- Queue complete frame to DMA buffer for transmission

Layer 3: ARQ Layer (Custom Code)

RX Path:

- Check sequence number against `last_acked_rx`
- If duplicate (`seq ≤ last_acked_rx`) → send ACK, discard payload
- If expected (`seq = last_acked_rx + 1`) → accept, send ACK
- If out-of-order → send NACK, request retransmission

TX Path:

- Assign `tx_sequence_number` to outgoing frame
- Store frame in retry buffer
- Start timeout timer (e.g., 10ms)
- On timeout → retransmit (up to 3 attempts)
- On ACK received → clear retry buffer, increment `tx_seq`

Layer 4: Protobuf (nanopb library)

- Decode incoming bytes → `SetVelocityRequest`, `EmergencyStopRequest`
- Encode outgoing structs → `SetVelocityResponse`, `TelemetryData`
- Uses `star.v1` package following Boston Dynamics style guide
- Static allocation (no malloc) with `.options` file constraints

Layer 5: Application

- Handle motor velocity commands
- Read encoder data via multiplexer
- Execute PID control loop (250 Hz per motor)
- Send telemetry responses (encoder feedback, current, temperature)

1.3.3 RPi5 (SPI Controller) Implementation

Layer 1: SPI Controller with DMA

- Use Linux `/dev/spidev` interface
- Configure SPI3: 10 Mbps, mode 0, 8-bit words
- Use `ioctl(SPI_IOC_MESSAGE)` for DMA transfers
- Implement userspace double-buffering for continuous operation

Layer 2: Frame Layer (*Custom Code - SAME AS RX72N*)

RX Path: Parse frame header, verify CRC-32, extract payload or discard on CRC failure

TX Path: Build frame [`Header` | `Payload` | `CRC32` | `Footer`], calculate CRC-32, send via SPI

Layer 3: ARQ Layer (*Custom Code - SAME AS RX72N*)

RX Path: Check sequence number, detect duplicates → ACK and discard, detect out-of-order → NACK

TX Path: Assign sequence number, store in retry buffer, timeout and retry logic, handle ACKs

Layer 4: Protobuf (Go with protoc-gen-go)

- Encode: `SetVelocityRequest`, `EmergencyStopRequest`
- Decode: `SetVelocityResponse`, `TelemetryData`, `EncoderData`
- Uses `star.v1` package following Boston Dynamics style guide

Layer 5: Application (star-gateway)

- Send motor velocity commands from Nav2 (autonomous) or UI (manual)
- All control flows through Nav2 stack for consistent behavior
- Receive encoder feedback for odometry calculations
- Bridge ROS2 topics to/from RX72N via SPI

1.3.4 ARQ Symmetry and State Management

The ARQ layer uses **Stop-and-Wait** protocol (also called alternating bit protocol). This means each side sends one frame at a time, waits for acknowledgment (ACK) with a timeout, and retries up to 3 times before declaring failure. This approach provides:

- Simple implementation suitable for embedded systems
- Predictable worst-case latency for real-time control
- Low memory overhead (single retry buffer, not a window of frames)
- Sufficient throughput for 10 Mbps SPI with <1ms frame time

The ARQ protocol is **symmetric** — both sides implement identical Stop-and-Wait logic but maintain separate state for their own transmissions and receptions.

Each Side Maintains:

- **TX sequence number** (`tx_seq`): For packets it sends
- **RX last acknowledged** (`rx_last_ack`): For packets it receives
- **Retry buffer**: Stores unacknowledged outgoing packets
- **Timeout timer**: Triggers retransmission if no ACK received

Responsibility	RX72N (Peripheral)	RPi5 (Controller)
Sends	Telemetry, responses	Commands, requests
TX sequence numbering	Own <code>tx_seq</code>	Own <code>tx_seq</code>
ACK/NACK generation	Sends ACK for RPi5	Sends ACK for RX72N
Retry logic	Retries if no ACK	Retries if no ACK
Duplicate detection	Checks RPi5's <code>seq</code>	Checks RX72N's <code>seq</code>

Table 5: ARQ role distribution between controller and peripheral

Important: The SPI controller/peripheral roles (Layer 1) are **hardware-level only**. The ARQ layer (Layer 3) treats both sides as equals — each can send, ACK, and retry independently.

1.3.5 Example Transaction: RPi5 Sends Velocity Command

1. **RPi5 Application:** Create `SetVelocityRequest` (left=1.0 m/s, right=1.0 m/s)
2. **RPi5 Protobuf:** Encode to binary (nanopb)
3. **RPi5 ARQ:** Assign `seq=42`, save to retry buffer, start 10ms timeout
4. **RPi5 Frame:** Build packet with header, CRC-32, footer
5. **RPi5 SPI:** DMA transfer to RX72N (controller initiates)
6. **RX72N SPI:** DMA receives packet in double-buffer
7. **RX72N Frame:** Verify CRC-32 → valid
8. **RX72N ARQ:** Check `seq=42 = rx_last_ack+1` → accept
9. **RX72N Protobuf:** Decode to `SetVelocityRequest` struct
10. **RX72N Application:** Update motor setpoints, execute PID control
11. **RX72N ARQ:** Build ACK packet with `ack_seq=42`
12. **RX72N Frame:** Wrap ACK in frame with CRC-32
13. **RX72N SPI:** DMA transmits ACK (peripheral responds)
14. **RPi5 SPI:** DMA receives ACK packet
15. **RPi5 Frame:** Verify CRC-32 → valid
16. **RPi5 ARQ:** Got ACK(42) → clear retry buffer, cancel timeout

Outcome: Command successfully delivered with guaranteed delivery via ARQ. If ACK was lost or CRC failed, RPi5 would retransmit after 10ms timeout.

1.4 ARQ Communication Flow

1.4.1 Normal Operation (No Retransmission)

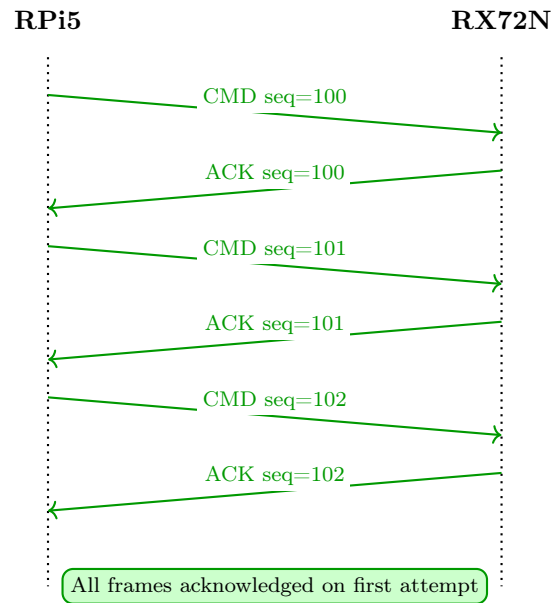


Figure 1: Normal ARQ Flow - Three Successful Transactions

1.4.2 Error Recovery Scenarios

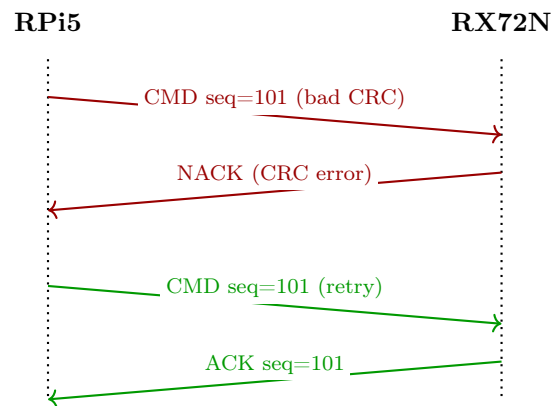


Figure 2: CRC Error - Retry with Same Sequence Number

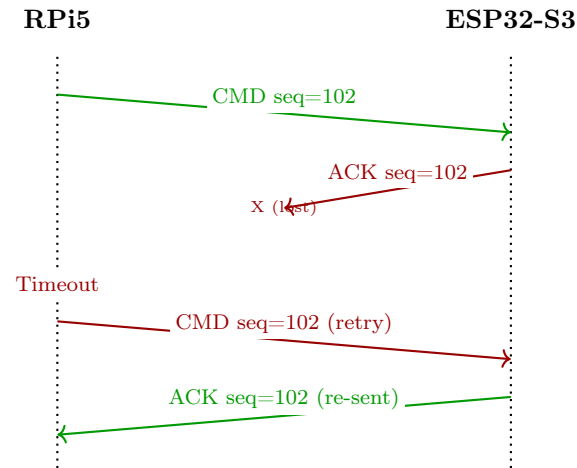


Figure 3: Duplicate Packet Scenario (Lost ACK)

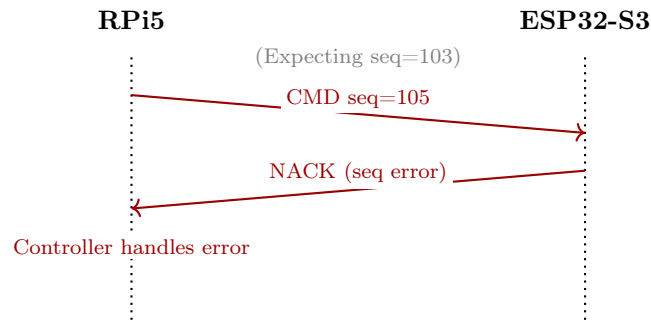


Figure 4: Out-of-Order Packet Scenario

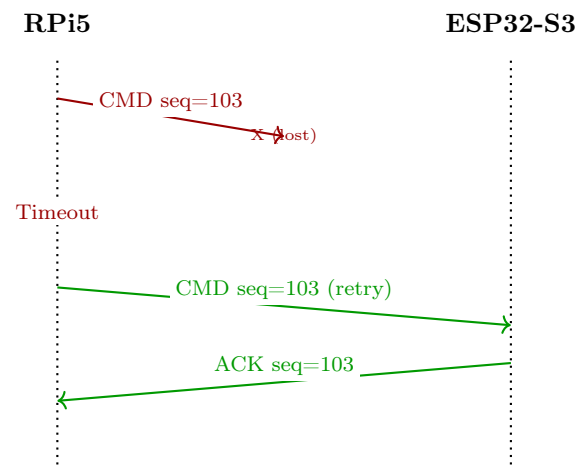


Figure 5: Lost Packet Scenario (Timeout)

1.5 Error Handling Strategies

CRC Mismatch

Action: NACK with CRC error reason
Recovery: Controller retry
Logging: Increment `crc_errors`

Sequence Error

Duplicate: Re-send ACK
Out-of-order: NACK + reject
Recovery: Controller retry

Timeout

RX timeout: Continue next cycle
500ms watchdog: Emergency stop
Retry fail: Log error

Invalid Protobuf

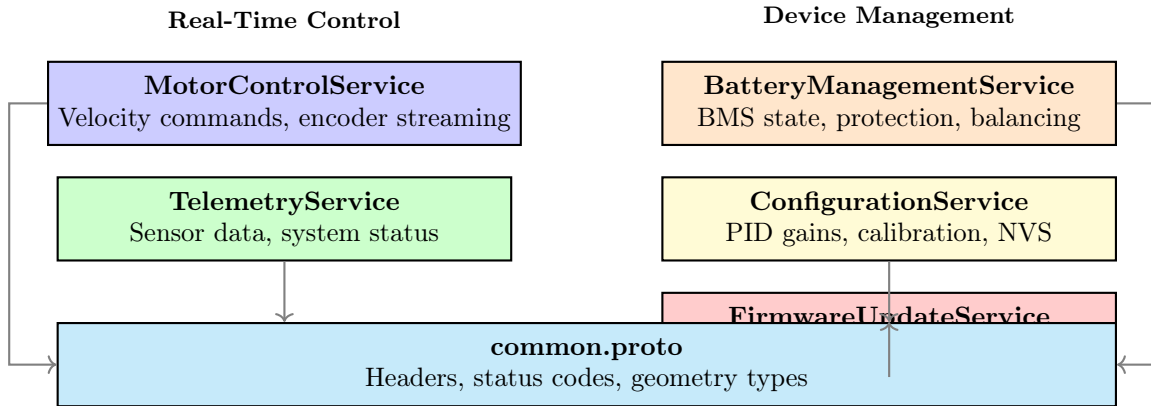
Action: NACK protocol error
Validate: IDs, ranges, offsets
Logging: Increment `proto_errors`

2 Protocol Buffer Schema Architecture

This section documents the Protocol Buffer schema architecture for RPi5↔RX72N communication. All schemas follow the **Boston Dynamics Protobuf Style Guide** for consistency, maintainability, and industry best practices.

2.1 Architecture Overview

Overview: Six gRPC services define the complete API surface for robot control, organized by functional domain. Services communicate via the nanopb-serialized frames described in Section 1.



Key Design Principles

- **Versioned Package:** All services use `star.v1` namespace for API evolution
- **Request/Response Headers:** Every RPC includes tracing, timestamps, and status
- **Streaming Support:** Continuous data uses server streaming (up to 100Hz)
- **Static Allocation:** All messages sized for nanopb constraints (no dynamic memory)

Note: Naming conventions, unit suffixes, and coding standards are consolidated in Section 4 (Style Guide and Conventions).

2.2 Service Specifications

2.2.1 MotorControlService

Purpose: Low-latency differential drive control with real-time encoder feedback.

Target Latency: <10ms round-trip for velocity commands.

Table 6: MotorControlService RPCs

RPC	Type	Description
SetVelocity	Unary	Set left/right wheel velocities (m/s)
EmergencyStop	Unary	Immediate stop, priority over all commands
SetMotorPower	Unary	Direct PWM control (bypass PID, testing only)
StreamEncoders	Server stream	Encoder ticks and velocity at 1-100Hz
ControlStream	Bidirectional	Velocity in, encoder feedback out

Key Messages:

- **VelocityCommand** – Left/right velocity (m/s), sequence number, timestamp
- **EncoderData** – Motor ID, tick count (341.2 PPR), velocity, timestamp
- **MotorStatus** – Duty cycle, velocity, temperature, current, fault flags
- **PidConfig** – Kp, Ki, Kd gains with output saturation limits

2.2.2 TelemetryService

Purpose: Real-time sensor data aggregation and system health monitoring.

Table 7: TelemetryService RPCs

RPC	Type	Description
GetTelemetry	Unary	Snapshot of all sensor data
StreamTelemetry	Server stream	Continuous sensor updates at 1-100Hz
GetSystemStatus	Unary	Firmware version, uptime, connection states

Key Messages:

- **TelemetryData** – IMU, GPS, battery %, WiFi signal, CPU usage, temperature
- **ImuData** – Pitch/roll/yaw (rad), acceleration (m/s^2), angular velocity (rad/s)
- **GpsData** – Lat/lon (degrees), altitude (m), accuracy, satellite count, fix type
- **SystemStatus** – Connection states, operating mode, free heap, uptime

Enumerations:

- **RobotMode** – IDLE, MANUAL, AUTONOMOUS, MAPPING, EMERGENCY_STOP
- **ConnectionStatus** – CONNECTED, DISCONNECTED, CONNECTING, ERROR
- **GpsFix** – NONE, 2D, 3D, DGPS, RTK_FLOAT, RTK_FIXED

2.2.3 BatteryManagementService

Purpose: BQ7850 BMS monitoring for lithium-ion battery pack safety and management.

Hardware: TI BQ7850 16-cell BMS with I2C interface.

Table 8: BatteryManagementService RPCs

RPC	Type	Description
GetBatteryState	Unary	Complete battery snapshot (cells, temps, SOC)
StreamBatteryState	Server stream	Continuous battery monitoring
Get/SetProtectionThresholds	Unary	OV/UV/OC/OT protection limits
Enable/DisableCellBalancing	Unary	Per-cell passive balancing control
ControlFets	Unary	Charge/discharge FET enable/disable
GetDeviceInfo	Unary	Manufacturer, serial, chemistry, versions

Key Messages:

- **BatteryState** – Composite of all battery data
- **CellData** – Individual cell voltages (mV), pack voltage, valid cell count
- **BatteryTemperatureData** – Thermistor readings (0.1°C), average
- **StateOfChargeData** – Relative/absolute SOC (%), capacity (mAh), cycle count
- **ProtectionThresholds** – OV/UV voltage limits, OC current limits, OT temperatures

2.2.4 ConfigurationService

Purpose: Runtime parameter management with NVS persistence across power cycles.

Table 9: ConfigurationService RPCs

RPC	Type	Description
GetConfiguration	Unary	Read current system configuration
SetConfiguration	Unary	Apply configuration (optional NVS persist)
ResetToDefaults	Unary	Restore factory defaults
ValidateConfiguration	Unary	Validate without applying
SaveConfiguration	Unary	Persist in-memory config to NVS
Get/SetMotorPidConfig	Unary	Per-motor PID tuning

Key Messages:

- **SystemConfiguration** – Complete config with version and CRC32
- **MotorPidConfig** – Per-motor PID gains (4 motors supported)
- **CurrentSensorCalibration** – Offset and scale for current sensing
- **SafetyThresholds** – Overcurrent, thermal shutdown, cell imbalance limits
- **TimingConfig** – Motor period, telemetry period, communication timeout

2.2.5 FirmwareUpdateService

Purpose: Over-the-air (OTA) firmware updates with validation and rollback support.

Chunk Size: 1KB optimal for SPI transfer efficiency.

Table 10: FirmwareUpdateService RPCs

RPC	Type	Description
BeginUpdate	Unary	Initialize OTA partition, set expected size/CRC
WriteChunk	Unary	Write single firmware chunk
StreamChunks	Client stream	Bulk chunk transfer
FinalizeUpdate	Unary	Validate firmware, prepare for reboot
AbortUpdate	Unary	Cancel and cleanup
Get/StreamUpdateProgress	Unary/Stream	Transfer progress and state
Reboot	Unary	Boot into new firmware
Rollback	Unary	Revert to previous firmware
MarkValid	Unary	Confirm firmware stability

Key Messages:

- **FirmwareChunk** – Data bytes, sequence number, offset
- **FirmwareUpdateProgress** – Total size, bytes received, state, CRC32
- **FirmwareInfo** – Version string, build date, git hash

Update States: IDLE → RECEIVING → VALIDATING → COMPLETE (or ERROR)

2.3 common.proto – Shared Types

All services import `common.proto` for standardized request/response handling.

Table 11: Standard Headers

Message	Fields
RequestHeader	request_id (UUID), client_timestamp, client_version, timeout
ResponseHeader	request_id (echo), server_timestamp, status, error_message, latency_us

Status Codes: UNKNOWN, OK, INVALID_REQUEST, INTERNAL_ERROR, NOT_FOUND, TIMEOUT, INVALID_STATE, ESTOP_ACTIVE

Geometric Types:

- **Vector3** – Generic 3D vector (x, y, z)
- **Quaternion** – Orientation (w, x, y, z)
- **SE2Pose** – 2D pose (x_m, y_m, angle_rad)
- **SE2Velocity** – 2D velocity (vx_mps, vy_mps, omega_rad_per_s)

2.4 Code Generation and Integration

Table 12: Code Generation Targets

Target	Generator	Output	Usage
Go	protoc-gen-go + grpc	gen/go/	star-gateway (RPi5)
nanopb	nanopb_generator	gen/nanopb/	RX72N firmware

nanopb Constraints: RX72N requires static allocation. Field sizes are configured in `.options` files:

- String fields: `max_size:64` (`request_id`), `max_size:128` (`error_message`)
- Repeated fields: `max_count:16` (cell voltages), `max_count:4` (motors)
- Bytes fields: `max_size:1024` (firmware chunks)

Build Integration:

- **Linting:** `buf lint` enforces style guide compliance
- **Breaking Changes:** `buf breaking` detects API incompatibilities
- **CI Pipeline:** Automatic generation and testing on PR

3 Hardware Pin Connections

3.1 RX72N Microcontroller Pinout (R5F572NNHGFP#30)

Hardware Advantage: The RX72N provides **182 direct GPIO pins** with dedicated peripheral functions. **No multiplexers or decoders are required** - each motor gets its own dedicated MTU channels for both PWM and encoder, with direct GPIO access to fault lines and current sense ADC channels.

3.1.1 Package Information

- **MCU:** Renesas RX72N (R5F572NNHGFP#30)
- **Package:** 100-pin LFQFP (Low-profile Quad Flat Package)
- **GPIO Pins:** 182 (multiplexed across package pins)
- **Flash:** 4 MB
- **RAM:** 1 MB
- **CPU:** RXv3 core at 240 MHz

3.1.2 Motor Control Pin Assignments

Direct Motor Control - No Multiplexing Required

Motor	PWM Channel	Encoder Channel	Fault Pin	Current Sense
Motor 0 (FL)	MTU3 (MTIOC3A/B)	MTU1 (Phase Count)	PORT3.2	AN000 (ADC)
Motor 1 (FR)	MTU4 (MTIOC4A/B)	MTU2 (Phase Count)	PORT3.3	AN001 (ADC)
Motor 2 (RL)	MTU6 (MTIOC6A/B)	MTU0 (Phase Count)	PORT3.4	AN002 (ADC)
Motor 3 (RR)	MTU7 (MTIOC7A/B)	MTU5 (Phase Count)	PORT3.5	AN003 (ADC)

Table 13: RX72N motor control pin assignments (FL=Front-Left, FR=Front-Right, RL=Rear-Left, RR=Rear-Right)

PWM Configuration:

- Frequency: 20 kHz (center-aligned PWM mode 1)
- Dead-time: 1 μ s for H-bridge safety
- Resolution: $\sim$ 12-bit (3000 counts at 120 MHz PCLKA)
- MTIOCA: PH (Phase) signal to DRV8243S
- MTIOCB: EN (Enable) signal to DRV8243S

Encoder Configuration:

- Mode: MTU Phase Counting Mode 1 (4x decoding)
- Inputs: MTCLKA (Phase A), MTCLKB (Phase B)
- Resolution: 341 PPR (Pulses Per Revolution)
- Counter: 16-bit hardware counter with software wraparound tracking

- Update Rate: Read at 250 Hz control loop

Fault Detection:

- Active-low nFAULT signals from DRV8243S
- Direct GPIO input (no external hardware needed)
- Polled at 250 Hz control loop

Current Sensing:

- IPROPI analog output from DRV8243S
- 12-bit S12ADFa ADC with PWM synchronization
- Range: 0-3.3V (0-2000 mA motor current)
- Sample time: 24 states for low-noise acquisition

3.1.3 Communication Interfaces

Debug UART (SCI1):

- PORT2.6: TXD1 (Transmit)
- PORT2.5: RXD1 (Receive)
- Baud Rate: 115200 bps
- Purpose: Debug output, logging, firmware update

SPI to Raspberry Pi 5 (RSPI0):

- PORT7.4: SS0A0 (Chip Select)
- PORT7.5: RSPCKA (Serial Clock)
- PORT7.6: MISOA (Controller In Peripheral Out - CIPO)
- PORT7.7: MOSIA (Controller Out Peripheral In - COPI)
- Mode: Peripheral (RX72N receives commands from RP5)
- Speed: 10 Mbps
- Protocol: nanosp + CRC-32

I2C Bus 0 (RIIC0) - BQ25798 Charger:

- SDA0, SCL0: I2C data and clock
- PORT5.0: BQ25798 INT (interrupt)
- Purpose: Battery charging control

I2C Bus 1 (RIIC1) - Fuel Gauge (SMBUS):

- SDA1, SCL1: SMBUS-compatible I2C
- PORT5.1: Fuel Gauge ALERT (interrupt)
- Purpose: Battery state-of-charge monitoring
- Features: Hardware CRC-8 PEC support

3.1.4 Status and Safety

Status LEDs:

- PORT0.3: Status LED (heartbeat)
- PORT0.5: Error LED (fault indication)
- PORT0.7: Power LED (system active)

Emergency Stop:

- PORT1.0: E-STOP button input
- Pull-up resistor, active-low
- Polled at 250 Hz, immediately stops all motors

3.1.5 System Clocks

Clock Domain	Frequency	Purpose
CPU (ICLK)	240 MHz	RXv3 core execution
PCLKA	120 MHz	MTU3a (PWM, Encoder)
PCLKB	60 MHz	CMT (Timers), SCI (UART)

Table 14: RX72N clock configuration

3.2 Raspberry Pi 5 Pinout

The Raspberry Pi 5 remains unchanged from the original design, handling high-level control, sensor fusion, and ROS2 communication.

Pin	RPi Label	Signal Name / Connection	Status / Note
1	3v3	Not Connected	Pi 3v3 Rail (Not used)
2	5v	Not Connected	Pi 5v Rail (Not used)
3	SDA	PI_SDA	I2C Data
4	5v	Not Connected	Pi 5v Rail (Not used)
5	SCL	PI_SCL	I2C Clock
7	IO4	HCSR04_TRIG1	Ultrasonic Trigger 1
8	TXD	LiDAR_RX_PI5_TX	UART Transmit to LiDAR
9	GND	GND	Common Ground
10	RXD	LiDAR_TX_PI5_RX	UART Receive from LiDAR
11	IO17	HCSR04_ECHO1	Ultrasonic Echo 1
12	IO18	HCSR04_TRIG4	Ultrasonic Trigger 4
13	IO27	HCSR04_TRIG2	Ultrasonic Trigger 2
15	IO22	HCSR04_ECHO2	Ultrasonic Echo 2
16	IO23	IMU_RST	IMU Reset
17	3v3	Not Connected	Pi 3v3 Rail (Not used)
18	IO24	IMU_INT	IMU Interrupt
19	COPI	RX72N_SPI_COPI	SPI Controller Out
21	CIPO	RX72N_SPI_CIPO	SPI Controller In
22	IO25	HCSR04_ECHO4	Ultrasonic Echo 4
23	SCLK	RX72N_SPI_SCLK	SPI Clock
24	CE0	RX72N_SPI_CS	SPI Chip Enable 0
26	CE1	HCSR04_TRIG5	Ultrasonic Trigger 5
27	ID_SD	MOTOR_DRIVER_MOTOR3_nFAULT	Motor 3 Fault
28	ID_SC	HCSR04_ECHO5	Ultrasonic Echo 5
29	IO5	MOTOR_DRIVER_MOTOR2_nFAULT	Motor 2 Fault
31	IO6	MOTOR_DRIVER_MOTOR1_nFAULT	Motor 1 Fault
32	IO12	HCSR04_TRIG6	Ultrasonic Trigger 6
33	IO13	MOTOR_DRIVER_MOTOR0_nFAULT	Motor 0 Fault
35	IO19	HCSR04_TRIG3	Ultrasonic Trigger 3
36	IO16	HCSR04_ECHO6	Ultrasonic Echo 6
37	IO26	HCSR04_ECHO3	Ultrasonic Echo 3
38	IO20	RX72N_SPI_DRDY1	SPI Data Ready 1
40	IO21	RX72N_SPI_DRDY2	SPI Data Ready 2

Table 15: Raspberry Pi 5 GPIO pin assignments

3.2.1 RPi5 Pin Function Categories

SPI (Controller Mode - Communication with RX72N):

- Pin 24 (CE0): Chip Select
- Pin 19 (COPI): Controller Out Peripheral In
- Pin 23 (SCLK): Serial Clock
- Pin 21 (CIPO): Controller In Peripheral Out
- Pin 38 (IO20): Data Ready 1 interrupt from RX72N
- Pin 40 (IO21): Data Ready 2 interrupt from RX72N

I2C (IMU and Stereo Cameras):

- Pin 3 (SDA): I2C Data

- Pin 5 (SCL): I2C Clock
- Pin 16 (IO23): IMU Reset
- Pin 18 (IO24): IMU Interrupt

UART (RPLiDAR C1):

- Pin 8 (TXD): LiDAR RX (Pi TX)
- Pin 10 (RXD): LiDAR TX (Pi RX)

Motor Driver Fault Lines (from RX72N):

- Pin 33 (IO13): Motor 0 nFAULT
- Pin 31 (IO6): Motor 1 nFAULT
- Pin 29 (IO5): Motor 2 nFAULT
- Pin 27 (ID_SD): Motor 3 nFAULT

HC-SR04 Ultrasonic Sensors (6 sensors):

- Sensor 1: Pin 7 (TRIG), Pin 11 (ECHO)
- Sensor 2: Pin 13 (TRIG), Pin 15 (ECHO)
- Sensor 3: Pin 35 (TRIG), Pin 37 (ECHO)
- Sensor 4: Pin 12 (TRIG), Pin 22 (ECHO)
- Sensor 5: Pin 26 (TRIG), Pin 28 (ECHO)
- Sensor 6: Pin 32 (TRIG), Pin 36 (ECHO)

3.3 Additional Peripheral Connections

3.3.1 IMU (Inertial Measurement Unit)

Hardware:

- BNO055: 9-axis absolute orientation sensor
- BMP280: Barometric pressure sensor

Protocol: I2C communication

Connection: Connects to RPi5 via Header J10:

- PI_SCL (Clock) routes to RPi5 Pin 5
- PI_SDA (Data) routes to RPi5 Pin 3
- IMU_RST routes to RPi5 Pin 16 (IO23)
- IMU_INT routes to RPi5 Pin 18 (IO24)

3.3.2 Stereo Cameras

Hardware: External stereo camera modules

Protocol: I2C communication

Connection: Connects via Header J12 to the same I2C bus (PI_SCL and PI_SDA), allowing the Pi to address them alongside the IMU using different I2C addresses.

3.3.3 Temperature Sensor

Hardware: DS18B20+ 1-Wire digital temperature sensor

Protocol: 1-Wire communication

Connection: Connected to RX72N GPIO (specific pin TBD in final hardware design)

4 Protocol Buffer Style Guide

This section defines naming standards and conventions for all Protocol Buffer definitions in the STAR project, following Boston Dynamics style guidelines.

4.1 Terminology Standard

The project uses inclusive terminology following OSHWA (Open Source Hardware Association) standards.

Table 16: Communication Bus Terminology

Use	Avoid	Context
Controller/Peripheral	Master/Slave	I2C, SPI, 1-Wire bus roles
COP1	MOSI	Controller Out, Peripheral In
CIP0	MISO	Controller In, Peripheral Out
Primary/Main	Master	Configuration structures

4.2 Protocol Buffer Naming Conventions (Boston Dynamics Style)

The STAR project follows Boston Dynamics' Protocol Buffer style guide for all .proto definitions.

Table 17: Protocol Buffer Naming Conventions

Element	Convention	Example
Package	Versioned namespace	<code>star.v1</code>
Messages	PascalCase	<code>VelocityCommand</code> , <code>BatteryState</code>
Fields	snake_case + unit suffix	<code>velocity_mps</code> , <code>temperature_celsius</code>
Enums	SCREAMING_SNAKE_CASE	<code>MOTOR_STATE_RUNNING</code>
Enum Zero Value	_UNKNOWN suffix	<code>STATUS_UNKNOWN = 0</code>
Enum Values	Type name prefix	<code>ROBOT_MODE_MANUAL</code>
RPC Pattern	{Verb}Request/Response	<code>SetVelocityRequest</code>

4.3 Unit Suffixes (MKS System)

All numeric fields use SI units with explicit suffixes. This eliminates unit ambiguity and enables automatic documentation generation.

Table 18: Standard Unit Suffixes

Suffix	Unit	Proto Example	C Example
<code>_m</code>	meters	<code>x_m</code>	<code>distance_m</code>
<code>_mps</code>	meters/second	<code>velocity_mps</code>	<code>speed_mps</code>
<code>_mps2</code>	m/s^2	<code>accel_x_mps2</code>	<code>acceleration_mps2</code>
<code>_rad</code>	radians	<code>angle_rad</code>	<code>heading_rad</code>
<code>_rad_per_s</code>	rad/second	<code>omega_rad_per_s</code>	<code>angular_vel_rad_per_s</code>
<code>_celsius</code>	degrees C	<code>temperature_celsius</code>	<code>motor_temp_celsius</code>
<code>_deci_celsius</code>	0.1°C	<code>temp_deci_celsius</code>	(BMS native format)
<code>_us</code>	microseconds	<code>timestamp_us</code>	<code>elapsed_us</code>
<code>_ms</code>	milliseconds	<code>timeout_ms</code>	<code>period_ms</code>
<code>_ma</code>	milliamps	<code>current_ma</code>	<code>motor_current_ma</code>
<code>_mv</code>	millivolts	<code>cell_mv</code>	<code>pack_voltage_mv</code>
<code>_mw</code>	milliwatts	<code>power_mw</code>	<code>consumption_mw</code>
<code>_percent</code>	0–100%	<code>soc_percent</code>	<code>duty_cycle_percent</code>

4.4 General Documentation Standards

- Use Doxygen-compatible comments for all public APIs
- Document the “why” not just the “what”
- Keep configuration centralized in header files
- Avoid magic numbers – use named constants with comments
- Version hardware and firmware together using git tags

4.5 NASA Power of 10: Rules for Safety-Critical Code

This section documents the NASA/JPL Power of 10 coding rules developed by Gerard J. Holzmann in 2006 for safety-critical embedded systems. These rules prioritize verification and predictability over coding flexibility.

STAR Compliance Philosophy: The STAR project follows the Power of 10 rules, with one **intentional architectural deviation** for Dependency Inversion Principle (DIP) — a conscious trade-off for testability and modularity.

4.5.1 Compliance Status Legend

Symbol	Meaning
COMPLIANT	STAR follows this rule
INTENTIONAL DEVIATION	Architectural choice (DIP pattern)

4.5.2 Rule 1: Simplify Control Flow

Rule: Restrict all code to very simple control flow constructs — no `goto`, `setjmp/longjmp`, or recursion (direct or indirect).

Rationale: Creates predictable program structure for human understanding and static analysis. Recursion causes unbounded stack usage — dangerous in embedded systems.

STAR Compliance: COMPLIANT

```

1  /* COMPLIANT - No goto, no recursion */
2  esp_err_t star_motor_init(star_motor_handle_t* handle,
3                          const star_motor_config_t* config)
4  {
5      if (handle == NULL || config == NULL) {
6          return ESP_ERR_INVALID_ARG;
7      }
8
9      /* All control flow uses if/while/for */
10     esp_err_t ret = configure_timer(handle, config);
11     if (ret != ESP_OK) {
12         return ret;
13     }
14
15     return configure_gpio(handle, config);
16 }

```

4.5.3 Rule 2: Fixed Loop Upper-Bounds

Rule: All loops must have a fixed upper bound that static analysis tools can trivially prove.

Rationale: Prevents runaway code, guarantees termination, enables real-time deadline verification.

STAR Compliance: COMPLIANT

```

1  /* COMPLIANT - Fixed iteration count */
2  #define MAX_RETRIES (3)
3
4  for (uint8_t i = 0; i < MAX_RETRIES; i++) {
5      if (sensor_read() == ESP_OK) {
6          break;
7      }
8      vTaskDelay(pdMS_TO_TICKS(100));
9  }
10
11 /* COMPLIANT - Array with known size */
12 #define BQ7850_MAX_CELLS (16)
13
14 for (uint8_t cell = 0; cell < BQ7850_MAX_CELLS; cell++) {
15     read_cell_voltage(cell, &voltages[cell]);
16 }

```

4.5.4 Rule 3: No Dynamic Memory After Initialization

Rule: Prohibit malloc/free after initialization phase.

Rationale: Dynamic allocation causes unpredictable performance, memory leaks, fragmentation, and corruption.

STAR Compliance: COMPLIANT

Best Practice: Use pre-allocated static pools instead of runtime memory requests.

```

1  /* COMPLIANT - Static allocation with fixed limits */
2  #define MAX_ITEMS (8)

```

```

3 #define MAX_DESC_LEN (64)
4
5 typedef struct {
6     char items[MAX_ITEMS][MAX_DESC_LEN]; /* Static 2D array */
7     uint8_t item_count;
8 } static_pool_t;
9
10 /* Usage */
11 if (pool->item_count >= MAX_ITEMS) {
12     return ESP_ERR_NO_MEM;
13 }
14 strncpy(pool->items[pool->item_count], desc, MAX_DESC_LEN - 1);
15 pool->items[pool->item_count][MAX_DESC_LEN - 1] = '\0';
16 pool->item_count++;

```

4.5.5 Rule 4: Keep Functions Short

Rule: Functions should not exceed ~60 lines (one printed page, one statement per line).

Rationale: Represents single verifiable logical units; improves comprehension, review, and testability.

STAR Compliance: **MOSTLY COMPLIANT**

```

1 /* COMPLIANT - Single responsibility, short function */
2 esp_err_t star_drv8243_read_current(star_drv8243_handle_t* handle,
3                                     float* current_ma)
4 {
5     if (handle == NULL || current_ma == NULL || !handle->initialized) {
6         return ESP_ERR_INVALID_ARG;
7     }
8
9     int raw_value = 0;
10    esp_err_t ret = read_adc_channel(handle, &raw_value);
11    if (ret != ESP_OK) {
12        return ret;
13    }
14
15    *current_ma = (float)raw_value * handle->ki_propi / 1000.0f;
16    return ESP_OK;
17 }

```

4.5.6 Rule 5: Use Assertions Generously

Rule: Minimum two assertions per function; assertions must be side-effect-free.

Rationale: Acts as executable documentation verifying pre-conditions, post-conditions, and invariants.

STAR Compliance: **COMPLIANT**

Note: STAR uses explicit parameter validation instead of assertions for production code (assertions disabled in release builds).

```

1 /* COMPLIANT - Explicit validation (better than assert for production) */
2 esp_err_t star_pid_compute(star_pid_handle_t* handle, float setpoint,
3                             float measurement, float dt, float* output)

```

```

4 {
5  /* Pre-condition checks (effectively runtime assertions) */
6  if (handle == NULL) {
7      return ESP_ERR_INVALID_ARG;
8  }
9  if (!handle->initialized) {
10     return ESP_ERR_INVALID_STATE;
11 }
12 if (output == NULL) {
13     return ESP_ERR_INVALID_ARG;
14 }
15 if (dt <= 0.0f || dt > 1.0f) { /* Sanity check time delta */
16     return ESP_ERR_INVALID_ARG;
17 }
18
19 /* Function logic */
20 float error = setpoint - measurement;
21 handle->integral += error * dt;
22
23 /* Post-condition: integral within bounds */
24 if (handle->integral > handle->integral_max) {
25     handle->integral = handle->integral_max;
26 } else if (handle->integral < handle->integral_min) {
27     handle->integral = handle->integral_min;
28 }
29
30 *output = handle->kp * error + handle->ki * handle->integral;
31 return ESP_OK;
32 }

```

4.5.7 Rule 6: Declare Data at Smallest Scope

Rule: Minimize variable scope to reduce accidental corruption risks.

Rationale: Limits chances of accidental reference or corruption from other code.

STAR Compliance: **COMPLIANT**

```

1 /* COMPLIANT - Variables declared at minimal scope */
2 void process_sensors(void)
3 {
4     /* Loop variable declared in for statement */
5     for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
6         /* Temporary variable for single sensor */
7         float temp_c = 0.0f;
8         if (read_sensor(i, &temp_c) == ESP_OK) {
9             /* Process immediately, temp_c out of scope after block */
10            update_telemetry(i, temp_c);
11        }
12    }
13 }

```

4.5.8 Rule 7: Check All Return Values and Parameters

Rule: Validate all function returns and input parameters; treat warnings as errors.

Rationale: Forces explicit failure-condition handling rather than ignoring error codes.

STAR Compliance: **COMPLIANT**

```

1  /* COMPLIANT - All returns checked, all parameters validated */
2  esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
3                                     star_bus_config_t* config)
4  {
5      /* Parameter validation */
6      if (manager == NULL || config == NULL) {
7          return ESP_ERR_INVALID_ARG;
8      }
9
10     /* Check return value and propagate errors */
11     esp_err_t ret = star_bus_config_init(config);
12     if (ret != ESP_OK) {
13         ESP_LOGE(TAG, "Bus initialization failed: %s", esp_err_to_name(ret));
14         return ret;
15     }
16
17     /* Mutex operation checked */
18     if (xSemaphoreTake(manager->mutex, pdMS_TO_TICKS(1000)) != pdTRUE) {
19         ESP_LOGE(TAG, "Failed to acquire mutex");
20         return ESP_ERR_TIMEOUT;
21     }
22
23     /* Critical section */
24     add_to_list(manager, config);
25     xSemaphoreGive(manager->mutex);
26
27     return ESP_OK;
28 }

```

4.5.9 Rule 8: Limit Preprocessor Use

Rule: Restrict to header inclusion and simple macros; forbid token pasting, recursive macros, and incomplete syntactic units.

Rationale: Preprocessor complexity obscures operations, fooling developers and static analysis tools.

STAR Compliance: **COMPLIANT**

```

1  /* COMPLIANT - Simple constant macros */
2  #define MAX_RETRY_COUNT (3)
3  #define MOTOR_PWM_FREQ_HZ (20000)
4  #define TIMEOUT_MS (1000)
5
6  /* COMPLIANT - Simple inline functions (better than complex macros) */
7  static inline float voltage_mv_to_v(uint16_t mv)
8  {
9      return (float)mv / 1000.0f;
10 }
11
12 /* AVOID - Complex token-pasting macros */
13 // #define CREATE_HANDLER(name) handler_##name##_init() /* Too complex */

```

4.5.10 Rule 9: Restrict Pointer Use

Rule: Maximum one dereferencing level (*ptr OK, **ptr forbidden); no function pointers.

Rationale: Multiple indirection and function pointers make static data-flow analysis impossible.

STAR Compliance: **INTENTIONAL DEVIATION**

Intentional Architectural Deviation: STAR uses function pointers for **Dependency Inversion Principle (DIP)** — an industry-standard pattern for testability and modularity. This is a conscious trade-off for better architecture.

DIP Pattern (Intentional Function Pointer Use):

```

1  /* INTENTIONAL DEVIATION - DIP interface with function pointers */
2  typedef struct star_error_interface {
3      esp_err_t (*record_error)(void* ctx, esp_err_t error, const char*
4          message,
5          const char* file, int line, const char* func);
6      bool (*can_retry)(void* ctx);
7      esp_err_t (*reset_state)(void* ctx);
8      void* ctx; /* Implementation context */
9  } star_error_interface_t;
10
11 /* Why we keep this:
12  * 1. Enables mock implementations for unit testing
13  * 2. Allows swapping error handlers without recompilation
14  * 3. Follows SOLID principles (Dependency Inversion)
15  * 4. Industry-standard pattern (used in Linux kernel, RTOS, etc.)
16  */

```

4.5.11 Rule 10: Compile with Maximum Warnings

Rule: Enable all compiler warnings at highest pedantry; achieve zero-warning builds with static analyzers.

Rationale: Modern compilers catch bugs before runtime. Zero warnings, no excuses.

STAR Compliance: **COMPLIANT**

```

1  # platformio.ini - Compiler flags
2  build_flags =
3      -Wall                # Enable all warnings
4      -Wextra              # Extra warnings
5      -Werror              # Treat warnings as errors
6      -Wno-unused-parameter # Allow unused params in interface
7                          implementations
8      -Wno-missing-field-initializers

```

CI/CD Enforcement: Build fails on any compiler warning.

4.5.12 Summary: STAR Compliance Matrix

Table 19: NASA Power of 10 Compliance Status

Rule	Description	STAR Status
1	No <code>goto</code> /recursion	COMPLIANT
2	Fixed loop bounds	COMPLIANT
3	No dynamic memory	COMPLIANT
4	Short functions (<60 lines)	COMPLIANT
5	Use assertions	COMPLIANT
6	Minimal variable scope	COMPLIANT
7	Check all returns	COMPLIANT
8	Limit preprocessor	COMPLIANT
9	Restrict pointers	INTENTIONAL (DIP)
10	Maximum warnings	COMPLIANT

4.5.13 References

- Holzmman, Gerard J. *“The Power of 10: Rules for Developing Safety-Critical Code.”* IEEE Computer, 39(6), pp. 95-97, June 2006.
- JPL Institutional Coding Standard for the C Programming Language (JPL DOCID D-60411)
- MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems

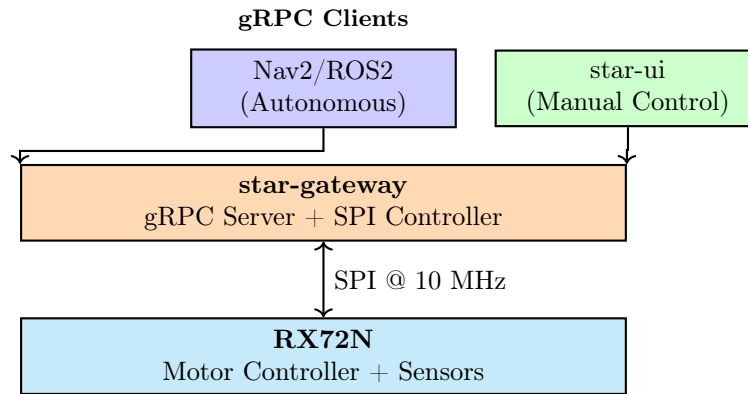
5 Gateway Architecture

This section documents the `star-gateway` Go service that bridges the Raspberry Pi 5 control stack with the RX72N motor controller.

5.1 Overview

The gateway service runs on the Raspberry Pi 5 and serves as the communication bridge between:

- **High-level side:** gRPC clients (Nav2/ROS2 for autonomous navigation, UI for manual control)
- **Low-level side:** RX72N motor controller via SPI at 10 MHz



5.2 Protocol Stack Implementation

The gateway implements Layers 1–4 of the communication protocol (see Section 1.2.1).

Table 20: Gateway Protocol Stack Implementation

Layer	Name	Go Package	Status
5	Application	<code>internal/service/</code>	Placeholder
4	Serialization	<code>star-proto/gen/go/</code>	Generated
3	ARQ	<code>internal/arq/</code>	Placeholder
2	Framing	<code>internal/frame/</code>	Implemented
1	Transport	<code>internal/transport/</code>	Placeholder

5.3 Directory Structure

```

star-gateway/
cmd/
  star-gateway/
    main.go          # Entry point
internal/
  transport/         # Layer 1: SPI transport
    spi.go           # Transport interface
    spi_test.go
  frame/             # Layer 2: Frame protocol
    frame.go         # Constants and types
  
```

```

    encoder.go          # Frame encoder
    decoder.go          # Frame decoder
    frame_test.go
  arq/                  # Layer 3: ARQ protocol
    arq.go              # Stop-and-Wait ARQ
    arq_test.go
  service/              # Layer 5: gRPC services
    motor_control.go
    telemetry.go
    battery.go
    configuration.go
    firmware.go
go.mod
CLAUDE.md

```

5.4 Frame Package

The `internal/frame` package defines the wire protocol constants and types.

5.4.1 Frame Constants

Table 21: Frame Protocol Constants

Constant	Value	Description
SyncWord	0x55AA	Frame sync marker (alternating bits: [0x55, 0xAA])
SyncSize	2 bytes	Sync word size
HeaderSize	6 bytes	SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1)
CRCSize	4 bytes	CRC-32 checksum
MaxPayloadSize	1024 bytes	Maximum protobuf payload
MaxFrameSize	1036 bytes	Total frame size
MinFrameSize	12 bytes	Empty payload frame

5.4.2 Frame Types

- `FrameTypeCommand` – Command from controller to peripheral
- `FrameTypeResponse` – Response from peripheral to controller
- `FrameTypeAck` – Acknowledgment of successful receipt
- `FrameTypeNack` – Negative acknowledgment (CRC error, etc.)

5.5 Transport Package

The `internal/transport` package provides the SPI transport layer.

5.5.1 SPI Configuration

Table 22: SPI Configuration Constants

Parameter	Value	Description
Device	/dev/spidev0.0	SPI device path
Speed	10 MHz	SPI clock frequency
Mode	0	CPOL=0, CPHA=0
Bits per word	8	Word size
Timeout	100 ms	Default operation timeout

5.5.2 Transport Interface

```

type Transport interface {
    Send(data []byte) (int, error)
    Receive(maxLen int) ([]byte, error)
    Transfer(txData []byte) ([]byte, error)
    Close() error
}

```

5.6 ARQ Package

The `internal/arq` package implements Stop-and-Wait ARQ for reliable delivery.

5.6.1 ARQ Parameters

Table 23: ARQ Protocol Parameters

Parameter	Value	Description
Max Retries	3	Maximum transmission attempts
Timeout	10 ms	ACK wait timeout
Sequence Max	0xFFFF	16-bit wraparound

5.6.2 ARQ State Machine

- **IDLE** – Ready to send or receive
- **WAITING_ACK** – Waiting for acknowledgment
- **RETRANSMITTING** – Retransmitting after timeout/NACK
- **ERROR** – Max retries exceeded

5.7 gRPC Services

The gateway exposes five gRPC services (see Section 2.2 for detailed specifications):

Table 24: Gateway gRPC Services

Service	Purpose
MotorControlService	Differential drive control, encoder streaming
TelemetryService	IMU, GPS, system status
BatteryManagementService	BQ7850 BMS monitoring
ConfigurationService	Runtime parameters, NVS persistence
FirmwareUpdateService	OTA updates, rollback

5.8 Build and Test

```
# Build the gateway binary
cd star-gateway
go build ./cmd/star-gateway
```

```
# Run all tests
go test ./...
```

```
# Run with verbose output
go test -v ./...
```

```
# Format code
go fmt ./...
```

```
# Static analysis
go vet ./...
```

5.9 Dependencies

The gateway depends on:

- github.com/Locked-Inc/star-proto/gen/go – Generated protobuf and gRPC code
- google.golang.org/grpc – gRPC framework
- google.golang.org/protobuf – Protocol Buffers runtime

The `go.work` file at the repository root enables workspace mode for seamless cross-module development:

```
go 1.21

use (
    ./star-proto/gen/go
    ./star-proto/tests/go
    ./star-gateway
)
```